

TestPlatform

Rapport d'Execution de Campagne

ms_gestion2

Projet: ms_gestion

Version: 2.5

Environnement: ms_gestion_env

Debut: 03/07/2026 15:03 | Fin: 03/07/2026 15:04

Duree: 92.2 s

Rapport genere le 03/07/2026 15:04

ECHEC

Resume executif

Total	Succes	Echecs	Erreurs	Duree
2	0	1	0	92.2 s

Taux de reussite



Evolution vs campagne precedente: DOWN 21.4 points

Campagne precedente #23

Corrections IA: 1 test(s) corrige(s) automatiquement, 0 avec succes

Informations du contexte

Projet	ms_gestion
Description	test

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main
Suite INTEGRATION	INTEGRATION	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
autha	UNIT	1	HIGH	IA
create envirenemnt	INTEGRATION	5	CRITICAL	IA

Environnement

URL Web	http://localhost:3000
URL API	N/A
Repository	https://github.com/ilyeschaabani/Automatisation_intelligente_des_tests_applicatifs
Branche	main
Base de donnees	POSTGRESQL

Resultats details

Synthese par suite

Suite	Total	Succes	Echecs	Taux
Suite UNIT	1	0	1	0%
Suite INTEGRATION	1	0	1	0%

Suite UNIT

autha		IA x3		FAILURE	
Type	UNIT	Duree	12.5 s		
Script	IA	Erreur	[COMPILATION_ERROR] COMPILATION ERROR :		

Suite INTEGRATION

create envirenemnt

Type	INTEGRATION	Duree	N/A
Script	IA	Erreur	[UNKNOWN] —

Analyse IA

autha

Erreurs extraites	<pre>[ERROR] COMPILATION ERROR : [ERROR] /C:/tmp/ms-execution/exec- 8758300152830098989/Backend/ms_gestion/src/test/java/suites/unit/AuthControllerTest.java:[13,47] cannot find symbol [ERROR] /C:/tmp/ms-execution/exec- 8758300152830098989/Backend/ms_gestion/src/test/java/suites/unit/AuthControllerTest.java:[36,13] cannot find symbol [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.14.1:testCompile (default-testCompile) on project ms_gestion: Compilation failure: Compilation failure: [ERROR] /C:/tmp/ms-execution/exec- 8758300152830098989/Backend/ms_gestion/src/test/java/suites/unit/AuthControllerTest.java:[13,47] cannot find symbol [ERROR] symbol: class SignInRequest [ERROR] location: package com.pfe.platform.ms_gestion.dto.request [ERROR] /C:/tmp/ms-execution/exec- 8758300152830098989/Backend/ms_gestion/src/test/java/suites/unit/AuthControllerTest.java:[36,13] cannot find symbol [ERROR] symbol: class SignInRequest [ERROR] location: class suites.unit.AuthControllerTest [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	**Cause** : Le symbole <code>`SignInRequest`</code> n'est pas trouvé car la classe est censée appartenir à <code>`com.pfe.platform.authenticationmicroservice.dto`</code>, mais Maven signale qu'il la recherche dans <code>`com.pfe.platform.ms_gestion.dto.request`</code> – indication d'un packaging incorrect dans l'import ou d'un décalage entre le code source et les imports. **Correction** : Vérifier la définition réelle de la classe <code>`SignInRequest`</code> (son packaging réel), puis corriger l'import en ligne 13 : si elle est dans <code>`com.pfe.platform.authenticationmicroservice.dto.request`</code>, changer <code>`com.pfe.platform.authenticationmicroservice.dto`</code> en <code>`com.pfe.platform.authenticationmicroservice.dto.request`</code>. **Conseil** : Toujours s'assurer que les imports correspondent exactement au packaging déclaré dans les fichiers sources (souvent via un clic droit <code>*Organize Imports*</code> dans l'IDE avant compilation).

create envirenemnt

Erreurs extraites	
Analyse IA	Analyse IA non disponible

Corrections automatiques par IA

L'IA a detecte des erreurs dans certains scripts generes et les a corriges automatiquement.

autha

3 correction(s)

FAILURE

```
=== Tentative 1/3 ===
Erreur détectée: [ERROR] COMPILATION ERROR :
[ERROR] /C:/tmp/ms-execution/exec-8758300152830098989/Backend/ms_gestion/src/test/java/suites/unit/AuthControllerTest.java:[3,62] package com.pfe.platform.authenticationmicroservice.Controller does not exist
[ERROR] /C:/tmp/ms-execution/exec-8758300152830098989/Backend/ms_gestion/src/test/java/suites/unit/AuthControllerTest.java:[4,59] package com.pfe.platform.authenticationmicroservice.Service does not exist
[ERROR] /C:/tmp/ms-execution/exec-8758300152830098989/Back
Résultat: Script corrigé (3153 chars), relance Maven...
Résultat: Encore en échec (exit=1).

=== Tentative 2/3 ===
Erreur détectée: [ERROR] COMPILATION ERROR :
[ERROR] /C:/tmp/ms-execution/exec-8758300152830098989/Backend/ms_gestion/src/test/java/suites/unit/AuthControllerTest.java:[17,26] package javax.servlet.http does not exist
[ERROR] /C:/tmp/ms-execution/exec-8758300152830098989/Backend/ms_gestion/src/test/java/suites/unit/AuthControllerTest.java:[31,13] cannot find symbol
[INFO] BUILD FAILURE
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.14.1:testCompile (default-testCompile) on proje
Résultat: Script corrigé (2954 chars), relance Maven...
Résultat: Encore en échec (exit=1).

=== Tentative 3/3 ===
Erreur détectée: [ERROR] COMPILATION ERROR :
[ERROR] /C:/tmp/ms-execution/exec-8758300152830098989/Backend/ms_gestion/src/test/java/suites/unit/AuthControllerTest.java:[5,48] cannot find symbol
[ERROR] /C:/tmp/ms-execution/exec-8758300152830098989/Backend/ms_gestion/src/test/java/suites/unit/AuthControllerTest.java:[28,52] cannot find symbol
[INFO] BUILD FAILURE
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.14.1:testCompile (default-testCompile) on project ms_gestion: Compilati
Résultat: Script corrigé (2994 chars), relance Maven...
Résultat: Encore en échec (exit=1).
```

Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
autha	Suite UNIT	12.5 s

Temps d'execution par type

Type	Duree moyenne
UNIT	12.5 s

Tests instables

Aucun test instable detecte

Recommandations

- autha:

****Cause**** : Le symbole `SignInRequest` n'est pas trouvé car la classe est censée appartenir à `com.pfe.platform.authenticationmicroservice.dto`, mais Maven signale qu'il la recherche dans `com.pfe.platform.ms_gestion.dto.request` — indication d'un paquetage incorrect dans l'import ou d'un décalage entre le code source et les imports.

****Correction**** : Vérifier la définition réelle de la classe `SignInRequest` (son paquetage réel), puis corriger l'import en ligne 13 : si elle est dans `com.pfe.platform.authenticationmicroservice.dto.request`, changer `com.pfe.platform.authenticationmicroservice.dto` en `com.pfe.platform.authenticationmicroservice.dto.request`.

****Conseil**** : Toujours s'assurer que les imports correspondent exactement au paquetage déclaré dans les fichiers sources (souvent via un clic droit **Organize Imports** dans l'IDE avant compilation).

Annexe - Scripts generes par IA

create envirenemnt

```
package suites.integration;

import com.pfe.platform.ms_gestion.dto.request.CreateEnvironmentRequest;
import com.pfe.platform.ms_gestion.dto.response.EnvironmentResponse;
import com.pfe.platform.ms_gestion.entity.Environment;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.ProjectMemberId;
import com.pfe.platform.ms_gestion.repository.CampaignRepository;
import com.pfe.platform.ms_gestion.repository.EnvironmentRepository;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.ProjectRepository;
import com.pfe.platform.ms_gestion.service.EnvironmentService;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.context.SecurityContextHolderImpl;
import org.springframework.test.annotation.Rollback;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.context.TestPropertySource;
import org.springframework.test.context.testng.AbstractTestNGSpringContextTests;
import org.springframework.transaction.annotation.Transactional;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

import java.util.Collections;

import static org.testng.Assert.*;

@SpringBootTest
@ActiveProfiles("test")
@Transactional
@Rollback(true)
@TestPropertySource(properties = {
    "spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;MODE=PostgreSQL",
    "spring.datasource.driverClassName=org.h2.Driver",
    "spring.datasource.username=sa",
    "spring.datasource.password=",
    "spring.jpa.hibernate.ddl-auto=create-drop",
    "spring.jpa.database-platform=org.hibernate.dialect.H2Dialect"
})
public class EnvironmentServiceIT extends AbstractTestNGSpringContextTests {

    @Autowired private EnvironmentService environmentService;
    @Autowired private ProjectRepository projectRepository;
    @Autowired private ProjectMemberRepository projectMemberRepository;
    @Autowired private EnvironmentRepository environmentRepository;
    @Autowired private CampaignRepository campaignRepository;

    private Long projectId;
    private Long userId = 1L;

    @BeforeMethod
    public void setUp() {
        // Insert prerequisite Project
        Project project = new Project();
        project.setName("AddProject");
        project.setDescription("Test project");
        project.setGitRepoUrl("https://github.com/org/webapp");
        project.setCreatedBy(userId);
    }
}
```

```
project = projectRepository.save(project);
projectId = project.getId();

// Insert ProjectMember to satisfy checkMembership
ProjectMember projectMember = new ProjectMember();
projectMember.setId(new ProjectMemberId(userId, projectId));
projectMember.setProject(project);
projectMemberRepository.save(projectMember);

// Set security context
SecurityContextHolder.setContext(new SecurityContextImpl(
new UsernamePasswordAuthenticationToken(userId, null, Collections.emptyList())));
}

@AfterMethod
public void tearDown() {
SecurityContextHolder.clearContext();
}

@Test
public void test_add_happy_path() {
// Given
CreateEnvironmentRequest request = new CreateEnvironmentRequest();
request.setName("AddProject");
request.setBaseUrlWeb("https://app.example.com");
request.setBaseUrlApi("https://api.example.com");
request.setGitRepoUrl("https://github.com/org/webapp");
request.setGitBranch("develop");
request.setDatabaseType("POSTGRESQL");

// When
EnvironmentResponse result = environmentService.add(projectId, request);

// Then
assertNotNull(result);
assertNotNull(result.getId());
assertEquals(result.getName(), "AddProject");
assertEquals(result.getBaseUrlWeb(), "https://app.example.com");
assertEquals(result.getBaseUrlApi(), "https://api.example.com");
assertEquals(result.getGitRepoUrl(), "https://github.com/org/webapp");
assertEquals(result.getGitBranch(), "develop");
assertEquals(result.getDatabaseType(), "POSTGRESQL");

// Verify persistence in DB
Environment savedEnv = environmentRepository.findById(result.getId()).orElse(null);
assertNotNull(savedEnv);
assertEquals(savedEnv.getName(), "AddProject");
assertEquals(savedEnv.getProject().getId(), projectId);
}
}
```

autha

```
package suites.unit;

import com.pfe.platform.authenticationmicroservice.Controller.AuthController;
import com.pfe.platform.authenticationmicroservice.Service.AuthenticationService;
import com.pfe.platform.authenticationmicroservice.dto.JwtAuthenticationResponse;
import com.pfe.platform.authenticationmicroservice.dto.RefreshTokenrequest;
import com.pfe.platform.authenticationmicroservice.dto.SignInRequest;
import com.pfe.platform.authenticationmicroservice.security.SecurityUtils;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.MockitoAnnotations;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
```

```
import javax.servlet.http.HttpServletResponse;
import java.util.Optional;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.*;
import static org.testng.Assert.*;

public class AuthControllerTest {

    @Mock
    private AuthenticationService authenticationService;

    @Mock
    private HttpServletResponse response;

    @InjectMocks
    private AuthController authController;

    private AutoCloseable mocks;
    private MockedStatic<SecurityUtils> mockedSecurity;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
        mockedSecurity = mockStatic(SecurityUtils.class);
        mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mockedSecurity.close();
        mocks.close();
    }

    @Test
    public void test_signIn_happy_path() {
        // Given
        SignInRequest request = new SignInRequest();
        request.setUsername("user.test");
        request.setPassword("SecurePass123!");

        JwtAuthenticationResponse authResponse = new JwtAuthenticationResponse();
        authResponse.setAccessToken("validAccessToken");
        authResponse.setRefreshToken("validRefreshToken");
        authResponse.setUsername("user.test");

        when(authenticationService.signIn(eq("user.test"), eq("SecurePass123!")))
            .thenReturn(authResponse);
        when(authenticationService.setAuthenticationCookies(any(HttpServletResponse.class), eq("validAccessToken"),
            eq("validRefreshToken")))
            .thenReturn(true);

        // When
        var result = authController.signIn(request, response);

        // Then
        assertNotNull(result);
        assertEquals(result.getStatusCode().is2xxSuccessful(), true);
        assertNotNull(result.getBody());
        assertEquals(result.getBody().getUsername(), "user.test");
        assertEquals(result.getBody().getAccessToken(), "validAccessToken");
        assertEquals(result.getBody().getRefreshToken(), "validRefreshToken");
        verify(authenticationService).signIn(eq("user.test"), eq("SecurePass123!"));
        verify(authenticationService).setAuthenticationCookies(eq(response), eq("validAccessToken"),
            eq("validRefreshToken"));
    }
}
```